

CAPSTONE PROJECT

(Submitted in partial fulfillment of the Summer Internship on Full Stack Web Development, 2025)

COCO-COLLABORATIVE CODE EDITOR

Presented By: Girisha Anamala,
Department and College Name: CSE-SRM University AP

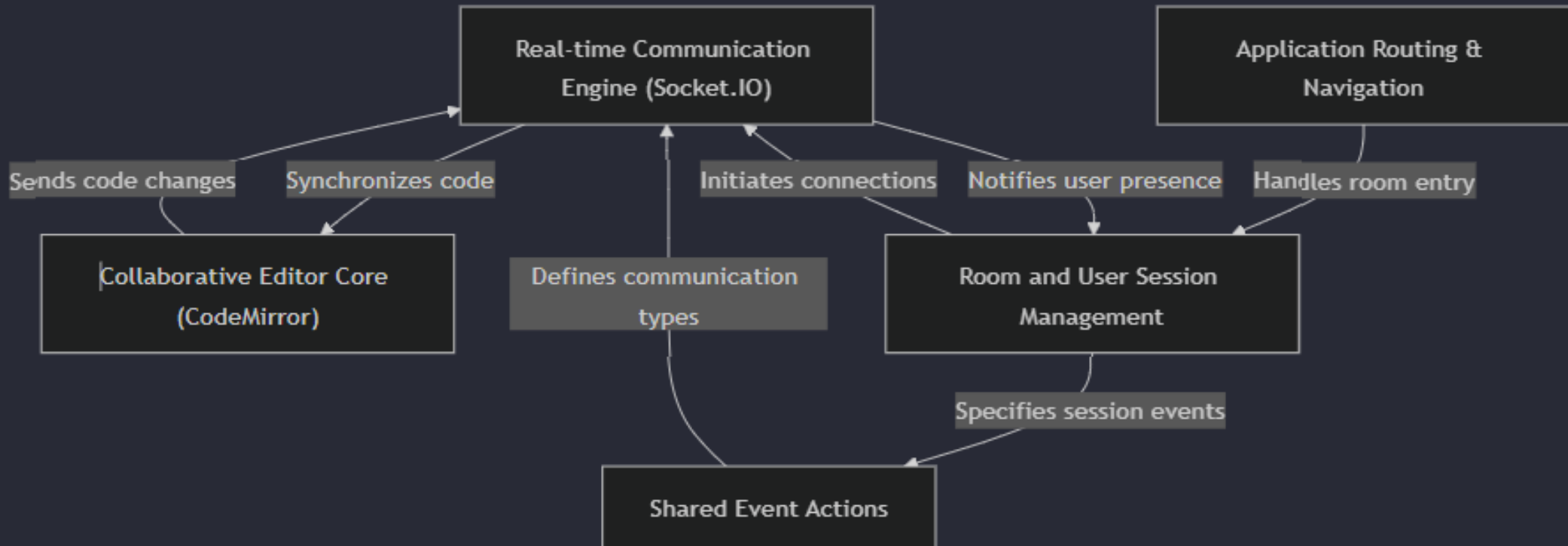
OUTLINE

- **Problem Statement**
- **System Architecture & Explanation**
- **Technology stack**
- **Database Schema & API Design**
- **Implementation & Key Features**
- **Deployment & Live Demonstration**
- **Results & Code (Sample snippets)**
- **Conclusion**
- **Future Scope**
- **References**

PROBLEM STATEMENT

- Developers, students, and coding teams face significant challenges when attempting to collaborate on code in real-time, especially in remote work environments.
- Many existing collaborative coding platforms are either **too complex for casual use**, require expensive subscriptions, or **demand extensive technical setup** that creates barriers for beginners, students, and small teams. This **lack of accessible, user-friendly solutions** prevents effective pair programming, code reviews, technical interviews, and educational coding sessions from happening seamlessly.

SYSTEM ARCHITECTURE



TECHNOLOGY STACK

- **Frontend: React** is used for building the user interface.

□ **Libraries:**

- React Router DOM for navigation
- CodeMirror for the collaborative code editor
- React Hot Toast for notifications
- React Avatar for user identification
- Socket.IO Client for real-time communication
- UUID for unique room/session generation

- **Backend:** The server-side is built with **Node.js** and the **Express.js framework**.

1. Node.js runtime environment
2. Express.js 4.17.3 for API and HTTP handling

1. APPLICATION ROUTING & NAVIGATION

Application Routing- Achieved through : React Router DOM

Setting the routes(src/app.js)

- 1. Home page
- 2. Editor page

When join is clicked we should route to editor page. When leave is clicked route back to home page

useParams- lets editor page know for which room id it should open for,
useLocation- is how the editor page get the username that was passed in home page

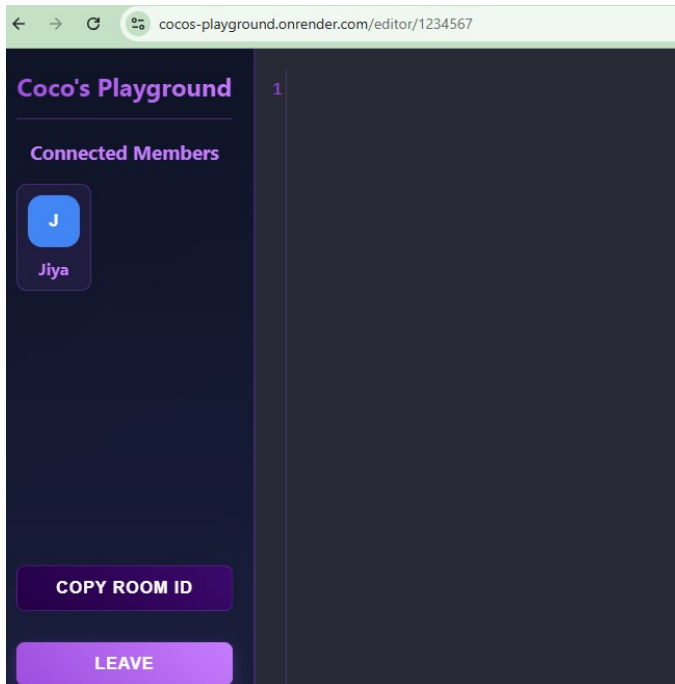
1.1. NAVIGATION BETWEEN PAGES (src/pages/home.js)

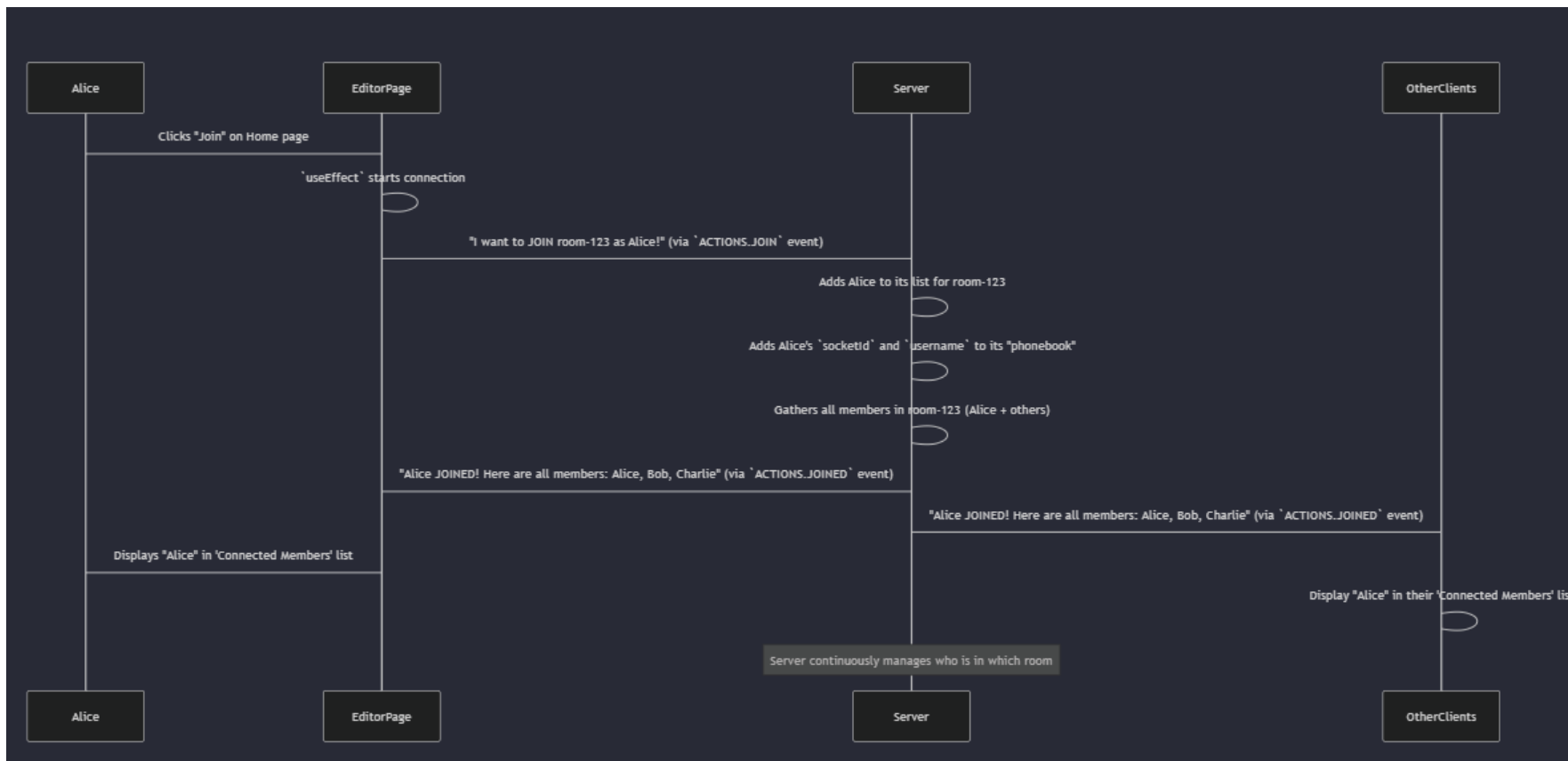
Now that we've defined our paths, how do we *move* from one page to another?
This is called **navigation**

We use a special hook from React-Router-DOM called useNavigate

`navigate('/editor/${roomId}')`

`{ state: { username, } }`





server.js

1. Phonebook of users- `userSocketMap`
2. `Socket.on(Actions.join..)-` listens for join message from client.
3. `Socket.in(RoomID).emit.Actions.Disconnected,{...})`
4. `delete.UserSocketMap[socket.id]`

Client side-Editor Page.js

- 1.Actions.join event- `socketRef.current.emit(ACTIONS.JOIN, { ... })` is sent by the client to the server requesting to join
`socketRef.current.on(ACTIONS.JOINED)`
`toast.success(...)`

2.

2. ROOM AND USER SESSION MANAGEMENT

USERS (CLIENTS)- EACH USER HAS:

1. **A USERNAME**
2. **SOCKET ID**

ROOMS- EACH ROOM IS IDENTIFIED BY A UNIQUE **ROOM ID**

SESSION MANAGEMENT- JOINING/ LEAVING AND LISTING MEMBERS

3. REAL-TIME COMMUNICATION ENGINE (SOCKET.IO)

- 1. **Instant Code Sync-** when you type it should appear in collaborators editor
- **2. Live User Lists** when people join/leave everyone in that room should be able to see their name appear and disappear
- **3. Notifications:** Small pop-up messages (like "Alice joined the room!") appear as events happen.
- HOW IT HAPPENS- SOCKET.IO
- 1. **Connecting to the Communication Engine-** src/socket.js
- **2.** It's a channel that allows both the client and the server to:
- **Send messages (emit events):** Like speaking into the chat channel.
- **Receive messages (listen for events):** Like hearing what others say in the chat channel.

Method	What it Does	Example Use Case
emit	Sends a message	Client sends <code>CODE_CHANGE</code> when you type.
on	Listens for a message	Client listens for <code>JOINED</code> when someone enters room.

1. Sending Messages from the Client (emit)-How Client uses this(editorpage.js)

```
1 // src/pages/EditorPage.js
2 // ... (inside useEffect after initSocket) ...
3
4 socketRef.current.emit(ACTIONS.JOIN, {
5   roomId, // The ID of the room we want to join
6   username: location.state?.username, // Our username
7 });
```



socketRef.current.emit(...): This tells our Socket.IO connection to *send* a message.



ACTIONS.JOIN: This is the *name* of the message (the event). It's a constant we defined in src/Actions.js to avoid typos.



{ roomId, username: ... }: This is the *data* we're sending along with the message.

```
1 // src/pages/EditorPage.js
2 // ... (inside useEffect after initSocket) ...
3
4 // Listening for the 'joined' event from the server
5 socketRef.current.on(
6   ACTIONS.JOINED,
7   ({ clients, username, socketId }) => {
8     if (username !== location.state?.username) {
9       toast.success(`${username} joined the room!`); // Show a pop
10    }
11    setClients(clients); // Update our list of connected members
12    // ... (code for syncing editor content for new joiners) ...
```

•**socketRef.current.on(...):** This tells our Socket.IO connection to *listen* for a specific message.

•**ACTIONS.JOINED:** This is the *name* of the message we're listening for.

•**({ clients, username, socketId }) => { ... }:** This is a function that will run *every time* the server sends an ACTIONS.JOINED message.

KEY SOCKET.IO SERVER METHODS IN SERVER.JS:

Method	What it Does	Example Use Case
<code>io.on('connection', ...)</code>	Listens for <i>new client connections</i> . Gets a <code>socket</code> object for each.	Our main entry point for client interactions.
<code>socket.on('eventName', ...)</code>	Listens for a specific <code>eventName</code> from <i>that particular client</i> .	When a client emits <code>ACTIONS.JOIN</code> or <code>ACTIONS.CODE_CHANGE</code> .
<code>socket.join(roomId)</code>	Adds a client's <code>socket</code> to a named <code>room</code> .	Essential for <u>Room and User Session Management</u> .
<code>socket.in(roomId).emit(...)</code>	Sends a message to <i>all clients in a specific room, except the sender</i> (<code>socket</code>).	Broadcasting code changes to collaborators without sending back to the person who typed.

Coco's Playground

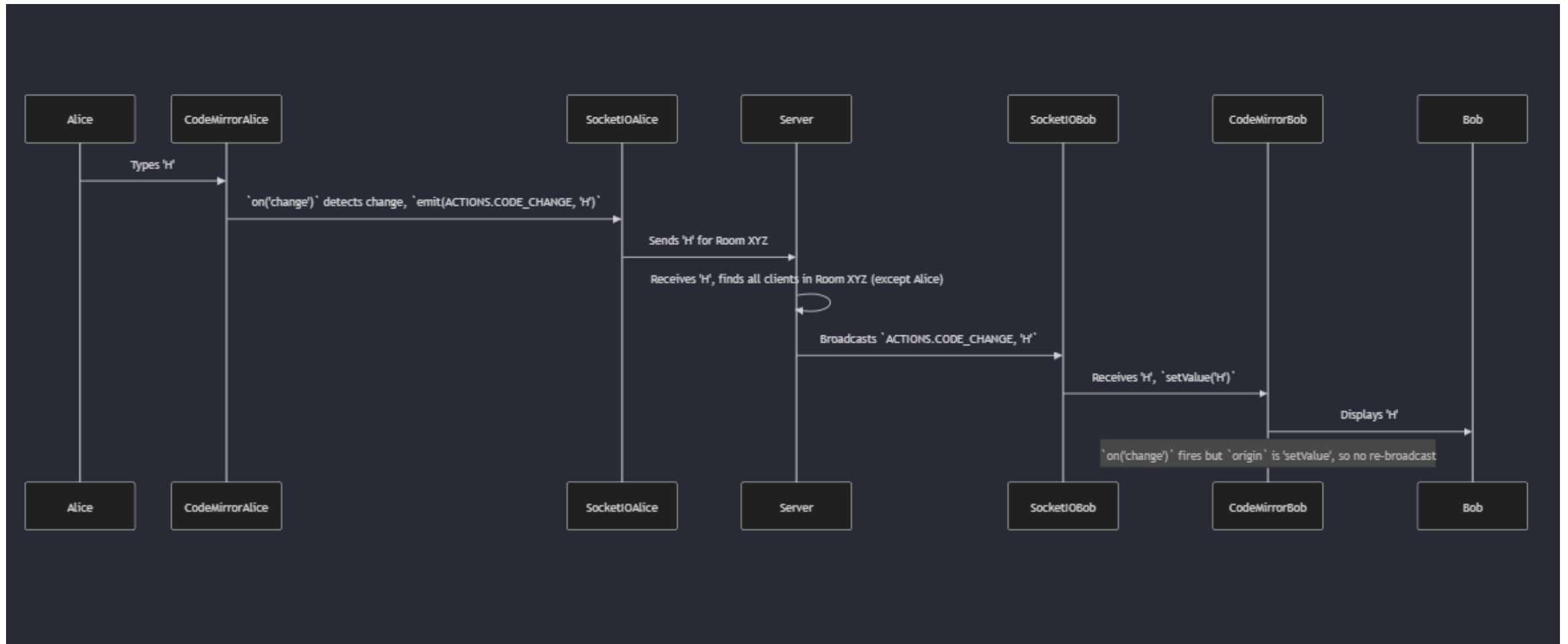
4. COLLABORATIVE EDITOR CORE (CODEMIRROR)

src/components/Editor.js

- When you type a letter in Coco's Playground, CodeMirror immediately tells our Real-time Communication Engine (Socket.IO) that a change happened.
- Socket.IO then broadcasts that change to everyone else in the room, and *their* CodeMirror instances update instantly.
- Key features of CodeMirror
 - 1. Syntax Highlighting:
 - 2. Line Numbers:
 - 3. Change Tracking-
 - it can detect and report *every single change* made by a user (a new character, a deletion, a paste).

```
1 #include<iostream>
2 int main(){
3     cout<<"hello Edunet";
4     return 0;
5 }
```

CODE MIRROR



IMPLEMENTATION & KEY FEATURES

■ **Project Setup and Environment Configuration**

- - Initialize React application using Create React App for frontend development
- - Set up Node.js backend with Express server configuration
- - Install and configure Socket.IO for real-time communication between client and server
- - Establish project structure with separate frontend (src/) and backend (server.js) components

IMPLEMENTATION & KEY FEATURES

❑ Frontend Development:

- - Implemented React Router for navigation between Home and Editor pages
- - Integrated CodeMirror component for collaborative code editing functionality
- - Designed and implemented space-themed UI with custom CSS animations and glassmorphism effects
- - Added real-time user presence indicators and notification systems using React Hot Toast
- **Home Page** Clean landing interface with room creation/joining functionality, featuring form inputs for Room ID and username entry with space-themed visual elements
- **Editor Page**: Main collaborative workspace built around CodeMirror editor
- **Navigation System**: React Router implementation for smooth transitions between pages without full page reloads

IMPLEMENTATION & KEY FEATURES

- ❑ **Backend Development:**
 - **Room Management** Dynamic creation and management of coding rooms
 - **User Session Handling** Track user connections, disconnections, and maintain active user lists for each room with automatic cleanup
 - **Real-time Synchronization:** Implement Socket.IO event handlers for code changes, cursor positions, and user presence updates
- **API Functionality**
 - **Connection Management:** Establish WebSocket connections with automatic reconnection handling and error recovery mechanisms
 - **Code Synchronization:** Broadcast code changes to all users in the same room while maintaining consistency and preventing conflicts
 - **User Authentication:** Simple username-based identification system without complex authentication requirements

IMPLEMENTATION & KEY FEATURES

Key Features

key feature is keeping all connected clients in sync in real-time.

Users can create a new room (with an auto-generated or manually entered Room ID) or join an existing room by entering a Room ID and username.

It allows users to copy the Room ID to invite others, and new users

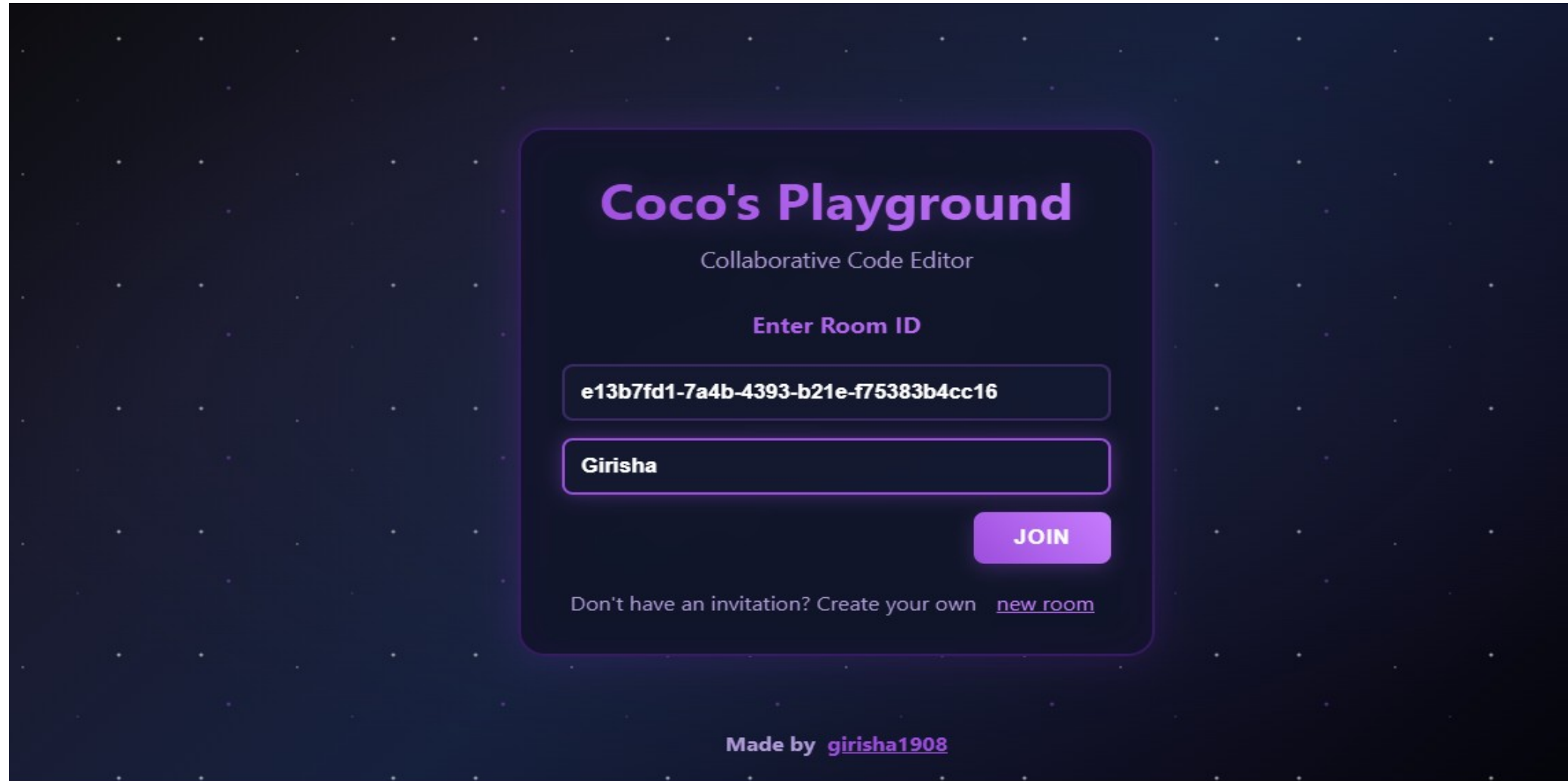
joining a room receive a notification and all existing code is synced.

Users can leave a room, which also triggers a notification and updates the connected users list.

DEPLOYMENT & LIVE DEMONSTRATION

- **Deployment:** I have used render to deploy my project.
- After deployment I updated the server.js code with the **Render URL** In the CORS settings.
- **Live Application Link:** <https://cocos-playground.onrender.com>

RESULTS AND CODE



RESULTS AND CODE

Coco's Playground

Connected Members



Girisha

COPY ROOM ID

LEAVE

```
1 #include<iostream>
2 int main(){
3     cout<<"Hello Girisha";
4 }
```

✓ Room ID copied to clipboard!

- **GitHub Repository:** <https://github.com/Girisha1908/Collaborative-Code-editor>

RESULTS AND CODE

The image shows two side-by-side browser windows. The left window, titled 'localhost:3000/e...', displays a web application interface. On the left side of this window, there is a sidebar with the title 'Coco's Playground'. Below the title, it says 'Connected Members' and shows a single member with a red square icon containing the letter 'G' and the name 'Girisha'. At the bottom of the sidebar are two buttons: 'COPY ROOM ID' and 'LEAVE'. The main area of the left window is a code editor showing the following C++ code:

```
1 #include<iostream>
2 int main(){
3     cout<<"hello Edunet";
4     return 0;
5 }
```

The right window, titled 'localhost:3000', shows the same 'Coco's Playground' interface but from a different perspective. It features a large central box with the title 'Coco's Playground' and the subtitle 'Collaborative Code Editor'. Below this, it prompts the user to 'Enter Room ID' and shows a text input field containing the room ID '5148af12-b1f4-45ff-b11b-3d729fabe856'. Another text input field contains the name 'Edunet'. A purple 'JOIN' button is positioned to the right of the name field. Below the input fields, there is a link that says 'Don't have an invitation? Create your own [new room](#)'. At the bottom right of the page, it says 'Made by [girisha1908](#)'. The word 'Paste' is written in large white text to the right of the room ID input field.

RESULTS AND CODE

Coco's Playground

Connected Members

G

E

Girisha

Edunet

COPY ROOM ID

LEAVE

```
1 #include<iostream>
2 int main(){
3     cout<<"hello Edunet";
4     return 0;
5 }
```

✓ Edunet joined the room!

Coco's Playground

Connected Members

G

E

Girisha

Edunet

COPY ROOM ID

LEAVE

```
1 #include<iostream>
2 int main(){
3     cout<<"hello Edunet";
4     return 0;
5 }
```

edunet
foundation

CONCLUSION

- **Primary Objective:** Created an accessible, real-time collaborative code editor that eliminates barriers to pair programming and remote coding collaboration.
- **User-friendly Interface** Developed a clean, space-themed UI making the platform accessible to beginners and professionals alike
- **Cross-platform Compatibility:** Built a responsive web application that works seamlessly across different devices and browsers without requiring installations
- **Problem faced 1-** Unstable connections and reconnection issues disrupted collaborative sessions
- **Solution:** Added robust connection handling with automatic reconnection, proper cleanup of disconnected users
- **Problem 2:** Local development settings conflicted with production environment requirements
- **Solution:** Implemented environment-based configuration using ``process.env`` variables for dynamic CORS origins, database connections, and server URLs

FUTURE SCOPE

1. Enhanced Code Editor Capabilities-

- **Monaco Editor Integration:** Replace CodeMirror with Microsoft's Monaco Editor (VS Code's editor) for advanced features like IntelliSense, error squiggles, and multi-cursor editing

2. Git Integration: Built-in version control with commit history, branch management, and conflict resolution directly within the collaborative editor

3. Video/Audio Communication: Will Integrate **WebRTC** for real-time voice and video calls during coding sessions, supporting screen sharing and collaborative debugging

4. Collaborative Debugging: Shared debugging sessions with breakpoint synchronization, variable inspection, and step-through debugging across multiple users

5. Whiteboard Integration: Will Add collaborative drawing canvas for system design discussions, algorithm visualization, and architectural planning



THANK YOU